

Backend Engineering Take-Home Assignment: Hotel Discovery API

Overview

As a software engineer at our company, you will help build core services that power travel experiences for millions of customers.

In this assignment, you are tasked with building a lightweight, production-ready RESTful API for finding and exploring hotels. Imagine you are building the backend for a travel platform where users need to efficiently filter through properties, dive deep into a specific hotel's details, and view what types of rooms are open for their travel dates. Your goal is to design a clean, maintainable, and well-structured service that demonstrates your command of API design, data modeling, and clean code principles.

 **Time Commitment:** We value your time and ask that you spend **no more than 3 hours** on this assignment. Focus on the core requirements first—we prefer a clean, working slice over an over-engineered, incomplete solution.

Technical Requirements

Your API should expose the following core resource endpoints. You may use the provided mock data to back these endpoints.

1. Search Hotels

- **Endpoint:** `GET /hotels` (or `GET /properties`)
- **Description:** Allows users to find hotels based on criteria.
- **Expectations:** Implement basic query parameters for filtering (e.g., `city`, `star_rating`, or `price_range`).

2. Get Hotel Details

- **Endpoint:** `GET /hotels/<id>`
- **Description:** Retrieves full detailed information for a specific hotel by its unique identifier.
- **Expectations:** Should return comprehensive data about the property, such as its name, address, description, amenities (e.g., pool, free Wi-Fi), and overall rating.

3. Get Room Availability & Pricing

- **Endpoint:** `GET /hotels/<id>/rooms`
- **Description:** Retrieves a list of available room types and live pricing for a specific hotel based on dates.
- **Expectations:** Accept `check_in` and `check_out` date parameters. It should return available options (e.g., "Deluxe King Room", "Standard Queen") along with the price per night.

Note on Status Codes & Error Handling: All successful API paths should support appropriate `2xx` status code and only the necessary `4xx` status code. Robust error handling is **not** a strict requirement for this assignment. However, if your API does return more errors, they should be well-documented in your submission explaining the "what" and the "why" behind them.

Deliverables & Guidelines

- **Language, Frameworks, & Libraries:** You are free to select **any language, frameworks, modules, or third-party libraries** you want to use. Pick the tools that best showcase your skills.
 - *Note: We use TypeScript/Node on our backend team, but this is entirely optional for the assignment—use what makes you comfortable.*
- **AI Tooling Policy:** We heavily welcome and encourage the use of AI tools (e.g., GitHub Copilot, ChatGPT, Claude) to help you build. The only catch is transparency: please include a quick section in your documentation explaining exactly how you leveraged AI during your coding and documentation process.
- **Documentation (`README.md`):** Include instructions on how to build, run, and test your application locally. Briefly explain your architectural choices and database design (if applicable).
- **Assumptions & Tradeoffs:** Add a separate documentation file that explains any tradeoffs or assumptions you made while writing your code. This is purely to help our engineering reviewers understand your thought process, not something you will be negatively judged against. Be as detailed as you like, but keep it contained to a single file.
- **Code Quality:** We value readable, well-organized code. Prioritize a clear separation of concerns, basic input validation, and rich, useful tests to demonstrate how you ensure your endpoints work as expected.
- **Submission:** Please upload your solution to a public github repository **no less than 24hrs before your interview** and share the URL with the Hiring Manager or Recruiter.

Mock Data Seed

To save you time from writing seed scripts, please use the included mock dataset to power your application. It contains a dynamic mix of 40 properties across different cities, price points, and ratings. Note that the data contains extra metadata fields beyond what is strictly required for the core endpoints—feel free to utilize them or ignore them as you see fit.